

AI Growth Automation System

Agentic Marketing Team: Full Development Brief

Version 2.0 | Start: April 28, 2026 | MVP deadline: June 30, 2026

0. How to Read This Document

This document is the single source of truth for building the Agentic Marketing Team. It defines every agent's identity, responsibilities, inputs, outputs, tools, documentation requirements, and phased development plan. Nothing should be assumed: if it is not written here, it does not exist yet.

The build sequence follows dependency logic. Each phase within an agent follows a confirm-then-expand rule: build the minimum, validate it works, then add the next layer. No agent moves to Phase B until Phase A is signed off by the intern responsible.

Key terms used throughout this document

Phase A (POC): Maximum 1 week. Minimum viable version on Telegram. Proves the concept works. Hardcoded logic, no advanced AI.

Phase B: Real intelligence layer. LLM-based decisions, real data, real API connections. ~3 weeks.

Phase C: Production-ready. Adaptive, multi-industry, Langfuse-instrumented, fully connected stack. ~2 weeks.

MVP: All agents at Phase B minimum, Commander and Qualifier at Phase C, full end-to-end funnel running on one real client.

Feature gate: A Phase B or C feature is only built if Phase A proved it was actually needed.

Intern sign-off: Each phase ends with a validation checklist. The intern must confirm every item before advancing.

Observability: Langfuse is introduced mid-Phase B (once LLM calls are real) and is mandatory for all agents in Phase C.

1. System Overview

1.1 What this system does

The AI Growth Automation System is a white-label agentic marketing engine. It replaces a full marketing team with six AI agents, each playing a senior specialist role. The system is deployed for clients across multiple industries: initially real estate, e-commerce, and healthcare: and is customised per client by the founding team before handoff.

The system runs the full funnel autonomously: generating paid traffic, qualifying leads, nurturing and booking appointments, analysing results, and feeding optimisation signals back into the ads.

1.2 The 6 agents

Acquisition layer

- **The Commander:** Orchestrator. Sits above all agents. Routes tasks, resolves conflicts, reports to operator. Built first: it is the skeleton everything plugs into.
- **The Media Buyer:** Manages paid advertising on Meta and Google. Owns targeting, budgets, bidding, and campaign structure.
- **Content & Conversion Strategist:** Produces all ad creatives, landing page copy, and form messaging. Owns CRO end to end.

Conversion layer

- **The Qualifier:** Runs dynamic intake forms, scores every lead 0–100, and classifies leads as MQL or SQL.
- **The Closer:** Handles all post-qualification communication: WhatsApp follow-up, email sequences, and appointment booking.

Intelligence layer

- **The CRM Keeper:** Data backbone. Writes every interaction to the CRM, maintains pipeline hygiene, serves clean data to all agents.
- **The Analyst:** Tracks ROI and ROAS. Identifies conversion patterns. Sends optimisation signals back to Commander and Media Buyer.

1.3 Full technology stack

Layer	Tool / Library	When introduced
Agent framework	LangChain	Phase A and B: all agents
Agent framework (adv)	DeepAgents / FastAgents / CrewAI	Phase C: Commander migrates first
Webhook server	FastAPI (Python): HTTP layer for external event intake	Phase A: all agents with external triggers
Python pkg manager	UV by Astral (replaces pip and poetry entirely)	From day one: all Python projects

Node pkg manager	pnpm (replaces npm for any JS tooling)	From day one: any JS tooling
LLM (lightweight)	Local SLM via Ollama (e.g. Phi-3, Mistral 7B)	Phase A: parsing, tagging, classification
LLM (mid tier)	DeepSeek API	Phase B: scoring, routing, reasoning
LLM (advanced)	OpenAI GPT-4o API	Phase B: copy gen, email, conversations
Observability	Langfuse : LLM tracing, evals, cost tracking	Mid Phase B: mandatory from Phase C
POC interface	Telegram Bot API	Phase A: all agents
WhatsApp	WhatsApp Business API via 360dialog or Twilio	Phase A: Closer only
CRM (mock)	Google Sheets : mock CRM for Phase A only	Phase A: CRM Keeper
CRM (production)	HubSpot or User.com API (behind an adapter class)	Phase B: CRM Keeper
Email delivery	SendGrid or Brevo API	Phase B: Closer
Calendar booking	Calendly or Cal.com API	Phase B: Closer
Ad platforms	Meta Ads API and Google Ads API	Phase A: Media Buyer (read-only)
Job scheduling	APScheduler (Python): daily and weekly jobs	Phase B: Analyst, Media Buyer

UV by Astral: the Python package manager for this project

UV replaces pip and poetry entirely. All Python dependency management uses UV from day one.

Setup: `curl -LsSf https://astral.sh/uv/install.sh | sh`

Create a project: `uv init agent-name`

Add a dependency: `uv add langchain langfuse openai`

Run a script: `uv run python main.py`

Lock file: `uv.lock` is committed to the repo. Never commit `requirements.txt`.

Why UV: 10-100x faster than pip, deterministic installs, built-in virtual environment management.

pnpm: the Node package manager for any JS tooling

Any JavaScript tooling (e.g. document generation scripts) uses pnpm, not npm.

Setup: `npm install -g pnpm` (one-time, then stop using npm).

Install deps: `pnpm install`

Add a dep: `pnpm add docx`

Why pnpm: faster installs, strict dependency isolation, smaller `node_modules`.

Langfuse: observability for LLM calls

Langfuse is introduced mid-Phase B: once agents are making real LLM calls and routing real leads.

Langfuse is MANDATORY from Phase C onwards. No Phase C agent is considered done without Langfuse tracing.

What Langfuse tracks: every LLM call (model, prompt, response, latency, cost), trace chains across agents, evaluation scores.

Self-host or cloud: use Langfuse Cloud for the POC and Phase B. Evaluate self-hosting before Phase C.

Integration: wrap every LLM call with the Langfuse Python SDK. Use `trace()` for agent-level spans, `generation()` for individual LLM calls.

Required metadata on every trace: `agent_name`, `client_id`, `lead_id` (if applicable), `phase`, `model_used`.

FastAPI: the webhook and HTTP layer for every agent

PURPOSE: FastAPI is the only way external events enter the system. It is NOT an automation runtime.

It does not orchestrate agents, it does not contain business logic, and it does not replace the agent framework.

It has exactly one job: expose HTTP endpoints so that external services (Typeform, WhatsApp, Meta Ads, Telegram, Google Ads) can send events INTO the system. Once an event arrives, FastAPI hands it to the Commander. From that point, the agent framework (LangChain, then DeepAgents/FastAgents/CrewAI) takes over.

WHAT REPLACES Make.com and n8n: Nothing. Those tools were runtime orchestrators. We do not need orchestration middleware because the agent framework handles inter-agent coordination natively. FastAPI replaces only the webhook ingestion part of what Make/n8n would have done.

ARCHITECTURE: One FastAPI app per agent that needs to receive external events.

- Commander: POST `/event` (receives all inbound events and routes them)
- Qualifier: POST `/form-submission` (receives Typeform webhook payload)
- Closer: POST `/whatsapp/incoming` (receives WhatsApp reply webhook)
- Media Buyer: POST `/ads/alert` (receives Meta/Google performance alerts)
- CRM Keeper: POST `/crm/update` (receives CRM-side change webhooks)

SETUP: `uv add fastapi uvicorn`

Run in dev: `uv run uvicorn main:app --reload`

Run in prod: `uv run uvicorn main:app --host 0.0.0.0 --port 8000 --workers 4`

IMPORTANT: Every FastAPI endpoint must validate the incoming payload before passing it to the agent.

Invalid payloads are rejected with a 422 response and logged. They never reach agent logic.
Every endpoint is documented with FastAPI's built-in OpenAPI schema (available at /docs).

Agent orchestration frameworks: LangChain, DeepAgents, FastAgents, CrewAI

These are NOT webhook servers. They handle agent-to-agent coordination, tool calling, memory, and task routing.

They are completely separate from FastAPI. FastAPI receives external events. The framework runs the agents.

Phase A and B: LangChain. Mature, well-documented, large ecosystem, works with all LLM providers.

Phase C: Migrate the Commander to DeepAgents, FastAgents, or CrewAI for production-grade multi-agent coordination.

DeepAgents: purpose-built for autonomous agent teams. Best fit if you want minimal configuration.

FastAgents: lightweight, fast startup, good for high-throughput agent pipelines.

CrewAI: role-based agent framework. Closest mental model to this system (agents as team members with defined roles).

Recommendation: evaluate CrewAI first. Its role/crew model maps directly to how this team is designed.

The migration from LangChain to any of these frameworks replaces the orchestration logic only.

FastAPI endpoints, LLM calls, CRM writes, and Langfuse traces are unaffected by the framework migration.

2. Build Order and Timeline

2.1 Reality check on timing

We are starting the last week of April. The MVP deadline is June 30. That is exactly 9 weeks. Phase A for every agent is capped at 1 week maximum: in most cases it should be 2 to 3 days. The goal of Phase A is not polish, it is proof. A working Telegram bot that does the one core thing is enough to move forward.

Phase B runs through May and early June. Phase C completes in the last two weeks of June. Any feature that is not done by June 30 is cut from the MVP and logged as a post-launch item.

2.2 Master timeline (9 weeks, April 28 → June 30)

Agent	Week 1 (Apr 28)	Weeks 2–4 (May)	Weeks 5–6 (Jun 1–15)	Weeks 7–8 (Jun 16–30)
1. Commander	POC shell + routing	Multi-agent coordination	Multi-client + override memory	Migrate to DeepAgents
2. CRM Keeper	Google Sheets mock CRM	Real CRM API + pipeline	Multi-CRM + segment sync	GDPR + archival jobs
3. Qualifier	Static form + scoring	Dynamic form + LLM score	A/B form + re-scoring	Learning from outcomes
4. Content & Conv. Str.	Ad copy gen (3 variants)	CRO + A/B management	Autonomous CRO cycle	Multi-language support
5. Media Buyer	Read-only API + alerts	Budget pacing + pausing	Bid adjustment + conv. API	Dayparting + rebalancing
6. Closer	WhatsApp first contact	Full sequences + booking	LLM replies + sentiment	Post-meeting follow-up
7. Analyst	— (logging only)	KPI dashboard + alerts	Attribution + Langfuse full	Predictive feedback loop

Important: Agents 1 and 2 (Commander and CRM Keeper) are built in parallel during week 1. All other agents can only start once both are at Phase A. Agents 3 and 4 (Qualifier and Content Strategist) also run in parallel from week 2.

2.3 Build sequence and dependency logic

1. Commander (week 1): built first. Everything else registers here. No other agent can be meaningfully tested without it.
2. CRM Keeper (week 1, parallel): built simultaneously. No agent can write or read lead data without it.
3. Qualifier (week 2): needs CRM Keeper to store scores. Needs Commander to route qualified leads.
4. Content & Conversion Strategist (week 2, parallel): no dependency on conversion layer. Runs alongside Qualifier.

5. Media Buyer (week 3): needs Commander for routing. Reads from Analyst eventually but can start without it.
6. Closer (week 3): needs Qualifier scores and CRM data. Cannot start before both are at Phase A.
7. Analyst (week 4): built last. Depends on data flowing from all other agents. Logging must be in place from day one.

Non-negotiable: structured logging from day one

The Analyst cannot function without historical data. Every agent must log every action from its first line of code.

Log every action as: { agent_name, action_type, input_summary, output_summary, lead_id, client_id, model_used, latency_ms, timestamp }

In Phase A: write logs to a local JSON file.

In Phase B: write logs to a central store (Supabase or a simple Postgres table).

In Phase C: all logs flow through Langfuse traces. The JSON log remains as a backup.

Interns: add the logging block FIRST before any other code in a new agent function. Not last.

3. Documentation Standards

Documentation is not optional. It is a deliverable equal in importance to working code. An agent that works but cannot be understood, maintained, or extended by another person is an incomplete agent. The following standards apply to every agent, every file, every function.

3.1 Docstrings: mandatory on every function

Every function, method, and class must have a docstring. No exceptions. The format is Google-style docstrings for Python. The docstring must explain what the function does, what it receives, what it returns, and what can go wrong.

Python docstring format (Google-style): use exactly this format

```
def score_lead(lead_data: dict, client_config: dict) -> dict:
```

```
    """
```

```
    Score a lead on a 0-100 scale using the weighted dimension model.
```

```
    Evaluates the lead across 5 dimensions (budget fit, timeline urgency,
    decision authority, problem-solution fit, engagement quality) and returns
    a total score, classification, and per-dimension breakdown.
```

```
    Args:
```

```
        lead_data: A lead record dict matching the CRM lead schema.
```

```
        Must contain: budget, timeline, role, problem_statement, form_answers.
```

```
        client_config: Client-specific config dict with scoring_weights and thresholds.
```

```
    Returns:
```

```
        dict: {
```

```
            "score": int (0-100),
```

```
            "classification": str ("disqualified" | "mql" | "sql"),
```

```
            "dimensions": dict (per-dimension scores),
```

```
            "explanation": str (human-readable 2-sentence summary)
```

```
        }
```

```
    Raises:
```

```
        ValueError: If lead_data is missing required fields.
```


LLMCallError: If the OpenAI API call for problem-solution fit fails.

""""

3.2 README per agent: mandatory

Every agent has its own directory in the repository. Every directory has a README.md. The README is the first thing an intern reads when assigned to an agent. It must be complete enough that a new person can understand, run, and extend the agent without asking anyone.

README.md required sections for every agent

Agent Name

One paragraph: what this agent does and its role in the system.

Responsibilities

Bullet list of what this agent owns. Copy from this brief.

What this agent does NOT do

Bullet list. Prevents scope creep.

Setup

Step-by-step: uv sync, environment variables needed, how to run locally.

Environment variables

Table: variable name | description | required | example value.

Running the agent

Exact commands to start the agent in dev mode and in production mode.

Running tests

Exact command: uv run pytest tests/

Current phase

Which phase this agent is currently at (A, B, or C) and what was completed.

Phase validation checklist

Copy the checklist from this brief. Check off items as they are validated.

Known limitations

Honest list of what the current phase cannot do. Updated every phase.

Inputs and outputs

What this agent receives (format + example) and what it produces (format + example).

3.3 Inline comments: when required

Interns decide when to add inline comments beyond docstrings, but the following cases always require an inline comment: no exceptions:

- Any non-obvious conditional logic: explain why the condition exists, not what it does.
- Any hardcoded value (threshold, timeout, limit): explain what it represents and where it came from.
- Any API call: explain what endpoint is being called and what the expected response shape is.
- Any error handler: explain what failure scenario it covers and why the chosen recovery strategy was selected.
- Any workaround or temporary fix: mark it with TODO: and explain what the permanent solution should be.

3.4 Repository structure: mandatory layout

Required directory structure for each agent	
agents/	
commander/	
README.md	← required
pyproject.toml	← UV project file
uv.lock	← committed, never edited manually
main.py	← entry point
api.py	← FastAPI app: defines all HTTP endpoints for this agent
agent.py	← core agent class
router.py	← routing/delegation logic
tools/	← LangChain tools, one file per tool
__init__.py	

telegram_tool.py	
tests/	
test_router.py	
test_agent.py	
.env.example	← all required env vars with placeholder values
CHANGELOG.md	← updated every time a phase is completed

3.5 CHANGELOG.md: required per agent

Every agent directory has a CHANGELOG.md. It is updated every time a phase is completed or a significant change is made. It does not need to follow a strict format: plain English is fine. The point is that anyone picking up the agent mid-development can see exactly what was done, when, and why.

Agent 1: The Commander

Orchestrator: routes, delegates, reports to operator

Layer: Above all layers

Stack: LangChain (A/B) then
DeepAgents/CrewAI (C) | FastAPI | UV |
Langfuse

Role definition

The Commander is the brain of the team. It does not do marketing work. Its job is to know which agent should handle what, at what moment, and to make sure the right information reaches the right agent. It acts as a senior account director: receives briefs, breaks them into tasks, delegates to specialists, and reports results back to you.

When a new lead enters the system, the Commander decides what happens to it. When a campaign underperforms, the Commander flags it. When two agents conflict, the Commander resolves it. When you need a status update, you ask the Commander via Telegram.

Responsibilities

- Receive all system events and route them to the correct agent.
- Maintain a live registry of all agents: status (idle, working, error), last action, last seen timestamp.
- Resolve inter-agent conflicts using the priority hierarchy defined below.
- Escalate to the operator when a situation exceeds its decision authority.
- Generate and send a daily digest to the operator at 6pm via Telegram.
- Store and retrieve the global system configuration per client (industry, CRM credentials, campaign IDs, scoring weights).

What the Commander does NOT do

- Write ad copy, score leads, send messages to prospects, or touch the CRM directly.
- Make media buying decisions: it instructs the Media Buyer, it does not execute them.
- Approve content: it routes content to the right place, it does not evaluate creative quality.

Conflict resolution priority hierarchy

8. Human operator instruction: always overrides everything.
9. Analyst performance data: overrides individual agent preferences.
10. Qualifier score: a score below 40 blocks the Closer from booking. No exceptions.
11. CRM Keeper data state: overrides any agent's in-memory state.
12. Commander's own LLM judgment: logged and flagged for operator review.

Phase A: POC (Week 1: April 28–May 2)

Goal: Prove the Commander can receive a Telegram message, parse it, and route it to the right webhook. Everything hardcoded. No LLM yet. Max 3 days to build, 2 days to validate.

Phase	Features to build
Phase A	Telegram bot that receives operator text commands and responds.
	Hardcoded routing table: keyword-to-agent webhook mapping stored in a config JSON file.
	Agent registry: JSON file listing all agents, their webhook URLs, and their current status. Status ping: Commander sends a health-check request to each agent and reports the result.
	Structured logging: every event logged to a local JSON file using the mandatory log format.
	FastAPI app: Commander exposes POST /event as the single entry point for all external events.
	Every incoming payload is validated by a Pydantic model before reaching any routing logic. Invalid payloads return 422 and are logged. FastAPI /docs endpoint is active and documents the /event schema automatically.
	README.md and .env.example created on day one before any logic is written.

Phase A validation checklist: intern must confirm all before moving to Phase B

Send 'new lead' via Telegram → Commander routes to Qualifier webhook and logs the event.

Send 'status' via Telegram → Commander returns all agent names and their last known status.

Send 'pause campaigns' → Commander sends the instruction to Media Buyer webhook.

Kill one agent webhook → status ping detects it as unreachable and reports correctly.

All events appear in log file with the correct mandatory structure.

README.md is complete and a second person can run the agent from it without help.

Phase B: Intelligence layer (Weeks 2–4: May)

Goal: Replace hardcoded routing with LLM-based intent classification. Add real multi-agent coordination and error recovery.

Phase	Features to build
Phase B	LLM-based intent parser using DeepSeek: Commander classifies incoming events dynamically, not by keyword.
	Priority queue: tasks ranked by urgency and lead score before dispatch. Agent health monitoring: if an agent fails to respond within 30 seconds, retry once then escalate to operator via Telegram.

	Conflict resolution engine: rule-based for the priority hierarchy, DeepSeek-assisted for edge cases. Every decision logged with reasoning.
	Client configuration store: Commander holds the active client config (industry, scoring weights, CRM credentials, campaign IDs). Loaded from environment on startup.
	Daily digest generation: GPT-4o generates a plain-language summary of all activity and sends it to operator at 6pm.
	Langfuse integration (mid-Phase B): wrap all LLM calls with Langfuse traces. Required metadata: agent_name='commander', client_id, action_type, model_used.

Phase B validation checklist

Send an ambiguous command ('something is wrong with leads') → Commander correctly interprets and routes.

Simulate an agent failure → Commander detects within 30 seconds and sends Telegram alert.

Run a full simulated lead journey (form → score → nurture → book) and verify Commander logs every step.
Daily digest arrives at 6pm with correct data.

Conflict test: Qualifier scores lead as 35, Closer tries to book → Commander blocks the booking.

Open Langfuse dashboard → see Commander LLM traces with correct metadata.

Phase C: Production (Weeks 7–8: June 16–30)

Goal: Full delegation, adaptive behaviour, multi-client support, full Langfuse observability.

Phase	Features to build
Phase C	Multi-client context switching: Commander holds separate configuration and state per client. Switching clients is a single config load.
	Operator override memory: Commander remembers past operator decisions and applies them to future similar situations without re-asking. Stored in a simple key-value store.
	Migration to DeepAgents or FastAgents: swap LangChain orchestration for production-grade multi-agent framework. Commander logic stays the same, the execution layer changes.
	Anomaly detection: Commander flags any metric deviating more than 20% from the 7-day rolling average. Alert sent immediately.
	Full Langfuse audit trail: every decision is a named trace with input, reasoning, output, and agent states at time of decision. Evaluations run weekly to score routing quality.
	Full docstring coverage verified: intern runs a docstring coverage check tool (e.g. interrogate) and achieves 100% coverage before Phase C sign-off.

Agent 2: The CRM Keeper

Data backbone: stores, retrieves, and maintains all lead data

Layer: Intelligence layer

Stack: LangChain | FastAPI |
HubSpot/User.com API | UV | Langfuse

Role definition

The CRM Keeper is the memory of the entire system. Every other agent is stateless by design: they process and act but do not remember. The CRM Keeper holds the truth. It knows every lead, what happened to them, their score, what messages were sent, and their current pipeline stage.

No agent reads from or writes to the CRM directly. All CRM operations go through the CRM Keeper. This is non-negotiable. It ensures data is validated before writing, schema consistency is enforced, and duplicate records are prevented.

Responsibilities

- Create a new lead record when a form is submitted. Assign a unique lead_id (UUID).
- Update lead records when any agent takes an action.
- Retrieve lead data on request from any agent. Return structured JSON only, never raw CRM objects.
- Enforce data schema: validate every field before writing. Reject malformed data with a descriptive error.
- Detect and merge duplicate leads using email and phone as deduplication keys.
- Maintain pipeline stage transitions in order: New, then MQL, then SQL, then Contacted, then Meeting Booked, then Closed Won or Closed Lost.
- Sync high-value segments back to ad platforms when the Analyst requests it.
- Archive leads inactive for more than 90 days via an automated nightly job.

Lead record schema: every field required unless noted

Mandatory lead record schema

lead_id: UUID. Generated at creation. Never changes.

client_id: Which client deployment this lead belongs to.

created_at: ISO 8601 timestamp of first form submission.

source: Campaign or channel ID that brought this lead (e.g. 'meta_campaign_abc123').

first_name, last_name: From form submission.

email: Validated email format. Primary deduplication key.

phone: E.164 format (e.g. +212661234567). Optional if email present.

industry_segment: Detected from form (real_estate / ecom / healthcare / other).

qualification_score: Integer 0–100. Null until Qualifier runs.

lead_stage: new / mql / sql / contacted / meeting_booked / closed_won / closed_lost.

last_action: String. Description of the most recent agent action on this lead.

last_action_at: ISO 8601 timestamp of last_action.

messages_sent: Integer. Count of total outbound messages across all channels.

meeting_datetime: ISO 8601. Null until Closer books. (Optional)

notes: Array of { author_agent, content, timestamp }. Append-only. Never overwrite.

Phase A: POC (Week 1: April 28–May 2)

Goal: Prove the CRM Keeper can create, read, and update lead records with correct validation. Backend is Google Sheets as mock CRM.

Phase	Features to build
Phase A	Google Sheets as mock CRM: one sheet = one client. Each row is a lead. Columns match the schema exactly. Telegram bot interface: operator sends JSON commands → CRM Keeper executes and responds. create_lead: validate required fields, check for duplicates by email/phone, write new row, return lead_id. get_lead: accept lead_id, return the full row as a validated JSON object. update_lead: accept lead_id + field updates, validate changes, update the correct row. Duplicate detection: before creating, search existing records. If email or phone matches, return existing lead_id with a warning. FastAPI app: CRM Keeper exposes POST /crm/update to receive CRM-side change webhooks from external tools. Every payload validated by a Pydantic model. Invalid payloads return 422 and are logged. All operations logged with mandatory log format. README.md and .env.example created on day one.

Phase A validation checklist

Create a lead via Telegram → row appears in Google Sheet with all required fields populated correctly.

Create the same lead again (same email) → system returns existing lead_id, no new row created.

Create a lead with a missing required field → system returns an error naming the exact missing field.

Update the lead stage → Google Sheet updates correctly and last_action_at is refreshed.

Get lead by lead_id → returns correct JSON matching the schema.

README.md is complete and runnable by a second person.

Phase B: Real CRM connection (Weeks 2–4: May)

Phase	Features to build
Phase B	Replace Google Sheets with real CRM API (HubSpot or User.com). Abstract behind a CRM adapter class so switching CRM = swapping the adapter only.
	Pipeline stage enforcement: stages can only advance (new → mql is valid, mql → new is blocked). Manual operator override allowed.
	Segment tagging: add and remove tags to leads based on Qualifier segments (e.g. tag: 'high_value_real_estate').
	Bulk retrieval: return all leads in a given stage or segment. Used by Closer (get all SQLs to contact) and Analyst (get all closed_won).
	Webhook receiver: listen for CRM-side changes (e.g. salesperson manually updates a stage) and sync back to system state.
	Langfuse integration (mid-Phase B): trace all CRM reads and writes. Tag with client_id and lead_id.

Phase C: Production (Weeks 7–8: June 16–30)

Phase	Features to build
Phase C	Multi-CRM support: different clients use different CRMs. CRM Keeper manages all via the adapter. Adding a new CRM = writing a new adapter class.
	Segment sync to Meta and Google Ads APIs: push high-value lead segments as Custom Audiences for retargeting. Triggered by the Analyst.
	Data quality scoring: flag leads with suspicious or incomplete data before they enter the pipeline.
	90-day archival job: automated nightly task moves inactive leads to archive and removes them from active segments.
	GDPR compliance flag: mark leads that have requested deletion. Block all future agent actions on flagged leads. Log the block.
	Full Langfuse audit trail: every CRM write is a named trace. Evaluations check data quality score distribution weekly.

Agent 3: The Qualifier

Lead intake, scoring 0 to 100, and MQL/SQL segmentation

Layer: Conversion layer

Stack: LangChain | FastAPI |
OpenAI/DeepSeek | Typeform API | UV |
Langfuse

Role definition

The Qualifier is the filter. Its job is to ensure only the right leads reach the sales process. A bad lead that reaches a salesperson wastes their time. A good lead filtered out is lost revenue. The Qualifier must be calibrated for each client's definition of a good lead.

The Qualifier runs three steps in sequence: intake (ask the right questions), score (evaluate answers against the weighted model), and segment (decide the lead's next destination). Its output is a scored, tagged, segmented lead record handed to the Commander for routing.

Scoring model: 5 dimensions, 0–20 each, total 0–100

Default scoring dimensions (adjustable per client in config)

1. Budget fit (0–20): Does the lead's stated budget match the client's minimum deal size? 0 = far below minimum. 20 = at or above target.
2. Timeline urgency (0–20): How soon does the lead want to act? 0 = just browsing, no timeline. 20 = decision this week.
3. Decision authority (0–20): Is the lead the decision maker? 0 = no authority (intern, student). 20 = owner or C-level.
4. Problem-solution fit (0–20): Does the lead's stated problem match what the client solves? Scored by LLM using a defined rubric.
5. Engagement quality (0–20): How complete and specific are their form answers? 0 = one-word answers. 20 = detailed and specific.

Classification thresholds

- 0–30: Disqualified. Tagged as disqualified. No further action. A polite rejection email is triggered automatically by the Closer.
- 31–60: MQL (Marketing Qualified Lead). Enters a 21-day nurturing email sequence. Not ready for a sales call.
- 61–100: SQL (Sales Qualified Lead). Flagged for immediate outreach. Closer attempts booking within 24 hours.

Phase A: POC (Week 1–2: April 28–May 5)

Goal: Prove the scoring model works end-to-end. Static form, hardcoded scoring logic, result written to CRM Keeper.

Phase	Features to build
Phase A	Telegram interface: operator sends a lead profile JSON → Qualifier returns a score, classification, and per-dimension breakdown.
	Static 5-question form sent via Telegram one message at a time. No branching logic yet.
	Hardcoded scoring: if/else rules per dimension. No LLM calls in Phase A: all rule-based. Output JSON: { lead_id, score, classification, dimensions, recommended_action }. Write score, classification, and dimensions to CRM Keeper (Google Sheets in Phase A).
	FastAPI app: Qualifier exposes POST /form-submission to receive Typeform webhook payloads directly. Payload is validated by a Pydantic model mapping Typeform field IDs to the lead schema before scoring starts.
	Log every scoring decision including the value given to each dimension and the rule that triggered it. README.md and .env.example on day one.

Phase A validation checklist

Submit a clearly strong lead (high budget, decision maker, urgent need) → score above 70, classified SQL.

Submit a clearly weak lead (student, no budget, just curious) → score below 30, classified Disqualified.

Submit an ambiguous lead → score between 31–60, classified MQL.

Check CRM Keeper → score, classification, and dimensions written correctly.

Log file shows per-dimension score breakdown for every submission.

README.md complete and runnable.

Phase B: Intelligence layer (Weeks 2–4: May)

Phase	Features to build
Phase B	Dynamic form via Typeform or LangChain form agent: questions adapt based on previous answers. Budget already stated → skip budget discovery question.
	LLM-scored dimensions: 'Problem-solution fit' and 'Engagement quality' scored by OpenAI GPT-4o with a defined rubric prompt.
	Industry-specific question sets: separate question flows for real estate, e-commerce, and healthcare. Selected from client config.
	Scoring weights configurable: stored in client config JSON. Changing a weight requires no code change.
	Disqualification trigger: Qualifier notifies Commander when score is below 30. Commander routes to Closer for rejection message.

Score explanation: GPT-4o generates a 2-sentence human-readable explanation of the score. Stored in CRM notes field.

Langfuse integration (mid-Phase B): trace every LLM scoring call. Tag with lead_id, client_id, dimension_name, model_used.

Phase C: Production (Weeks 7–8: June 16–30)

Phase	Features to build
Phase C	Learning from outcomes: Qualifier compares past SQL predictions against actual closed deals in CRM. Sends weight adjustment signals to itself. Logs the adjustment with reasoning.
	A/B form testing: two question flow versions run simultaneously. Qualifier tracks which version produces better SQL conversion rate. Real-time re-scoring: if a lead revisits the pricing page after being classified MQL, trigger a re-score automatically. Firmographic enrichment (B2B clients): auto-enrich lead with company size and revenue using Clearbit or Apollo.io API before scoring.
	Full Langfuse evaluation: weekly eval job scores the Qualifier's classification accuracy against CRM outcomes. Results visible in Langfuse dashboard.

Agent 4: Content & Conversion Strategist

Ad creatives, landing page copy, and full CRO ownership

Layer: Acquisition layer

Stack: LangChain | OpenAI GPT-4o | UV
| Langfuse

Role definition

The Content & Conversion Strategist owns everything a prospect reads or sees before they submit a form: ad copy, creative briefs, landing page copy and structure, form intro text, and thank-you page messaging. It also owns CRO: if a page is not converting, this agent identifies why and proposes the fix.

CRO ownership means: if a landing page converts below 15% of visitors, this agent flags it and proposes structural changes. If a form question causes more than 40% drop-off, this agent rewrites it. If an ad's CTR drops below 1%, this agent generates three new copy variants. The Media Buyer drives traffic. What happens on the page is entirely this agent's responsibility.

Responsibilities

- Generate ad copy variations: headline (max 40 chars), primary text (max 125 chars), call-to-action.
- Write creative briefs for visual assets (images, videos) that a designer or AI image tool can execute.
- Build and maintain landing page copy per client: all 6 required sections (hero, problem, solution, social proof, CTA, FAQ).
- Run A/B tests: generate two variants of any copy asset, assign IDs, track performance, flag winners.
- Analyse form drop-off data from the Qualifier and propose specific question rewrites.
- Receive performance data from the Analyst and rewrite underperforming assets proactively.
- Maintain a client swipe file: a versioned library of best-performing copy organised by type and platform.

Phase A: POC (Week 2: May 5–9)

Phase	Features to build
Phase A	Telegram interface: operator sends client brief (industry, offer, audience, tone) → agent returns 3 ad copy variants.
	Each variant: headline (≤40 chars), primary text (≤125 chars), one CTA selected from an approved list.
	Landing page copy template: operator sends client brief → agent returns full page copy in markdown covering all 6 sections.
	Output stored in a Google Doc per client with version number and timestamp.

	All outputs logged: { agent, client_id, asset_type, version, model_used, timestamp }.
	README.md and .env.example on day one.

Phase A validation checklist

Send a real estate brief → receive 3 distinct, on-brand ad copy variants with correct character counts.

Send a healthcare brief → receive 3 variants with appropriate tone (professional, trust-first).

Send an e-commerce brief → receive 3 variants with urgency and product-focused messaging.

Request landing page copy → receive all 6 sections in the correct order.

All outputs stored in Google Doc with correct client_id and version.

Phase B: CRO and A/B management (Weeks 2–6: May–early June)

Phase	Features to build
Phase B	Performance-aware rewriting: Analyst sends CTR and conversion data → agent automatically rewrites underperforming copy without operator request.
	A/B test management: generate two variants, assign IDs, track which is live, flag when enough data exists to declare a winner (minimum 100 impressions per variant).
	Form friction analysis: reads Qualifier drop-off data per question → proposes specific rewrites for high-drop-off questions.
	Industry tone profiles: calibrated tone per industry stored in config. Real estate = aspirational. Healthcare = reassuring. E-commerce = urgent.
	Swipe file management: every winning variant saved to client swipe file with performance metrics. Losing variants archived.
	Creative brief generation: structured brief (format, visual style, main message, do/don't list) ready for a designer or Midjourney/DALL-E.
	Langfuse integration (mid-Phase B): trace every GPT-4o generation call. Evaluate output quality weekly using a scoring rubric.

Phase C: Autonomous CRO (Weeks 7–8: June 16–30)

Phase	Features to build
Phase C	Autonomous CRO cycle: agent detects underperforming pages, generates variants, requests Commander to deploy them, monitors results, and iterates: no operator input required.
	Multi-language support: generate copy in French, Arabic, and English. Language selected from client config.
	Dynamic landing page personalisation: different copy for different audience segments (e.g. first-time buyers vs investors in real estate).

Full Langfuse evaluation: weekly eval scores copy output quality using a rubric. Low-scoring outputs flagged for operator review.

Agent 5: The Media Buyer

Paid advertising management and optimisation

Layer: Acquisition layer

Stack: LangChain | FastAPI | Meta Ads API | Google Ads API | UV | Langfuse

Role definition

The Media Buyer is responsible for getting the right people to see the right ad at the right cost. It manages all paid advertising across Meta (Facebook, Instagram) and Google (Search, Performance Max). It controls campaign structure, audience targeting, budget allocation, and bidding strategy.

The Media Buyer does not create ad copy or visuals. It receives finished assets from the Content & Conversion Strategist and deploys them. It uses performance data from the Analyst to continuously improve its decisions. Its primary success metric is not clicks: it is cost per SQL.

Responsibilities

- Create and manage campaign structures on Meta Ads and Google Ads per client.
- Configure audience targeting: Lookalike Audiences, interest-based audiences, CRM retargeting segments.
- Allocate and pace budgets across campaigns based on performance and client monthly caps.
- Monitor daily KPIs: CPM, CPC, CTR, CPL, cost per SQL, ROAS.
- Pause underperforming ad sets and reallocate budget to winners.
- Upload new creative assets from the Content & Conversion Strategist and deploy them into active campaigns.
- Verify conversion tracking via Meta Pixel and Google Ads conversion events.
- Send structured performance data to the Analyst every day.

Phase A: POC (Week 3: May 12–16)

Goal: Prove the agent can read real campaign data and alert on threshold breaches. Read-only. No writes to ad platforms yet.

Phase	Features to build
Phase A	Read-only API connection to Meta Ads and Google Ads: pull campaign metrics for the last 7 days.
	Daily metrics report: at 8am every day, pull yesterday's spend, impressions, clicks, leads per campaign. Send formatted summary to Telegram.
	CPL alert: if cost per lead exceeds a client-defined threshold, send Telegram alert immediately.
	Manual campaign summary: operator sends a campaign ID → agent returns its current settings and last 7-day performance.

	FastAPI app: Media Buyer exposes POST /ads/alert to receive performance alert webhooks from Meta and Google.
	Validated by Pydantic model. Alerts that do not match the expected schema are rejected and logged.
	All API calls logged with { platform, campaign_id, action, response_summary, timestamp }.
	README.md and .env.example on day one.

Phase A validation checklist

Connect to a real Meta Ads account → pull last 7 days of data correctly.

Connect to a real Google Ads account → pull last 7 days of data correctly.

Receive daily metrics report at 8am with correct numbers verified against the platform UI.

Set a CPL threshold → manually spike a metric above it → Telegram alert arrives within 5 minutes.

Phase B: Active management (Weeks 3–6: May–early June)

Phase	Features to build
Phase B	Budget pacing: daily check of spend vs days remaining in month. Alert if pacing is off by more than 15%.
	Automatic ad set pausing: if an ad set's CPL exceeds 2x the target for 3 consecutive days, pause it and notify operator.
	Lookalike Audience creation: when CRM Keeper pushes a new high-value segment, create a Lookalike Audience from it on Meta automatically.
	Creative rotation: when Content Strategist generates new variants, deploy them at 80% traffic. Old variants get 20% for comparison.
	Conversion event verification: confirm form submission fires the correct conversion event on Meta Pixel and Google Ads. Alert if firing stops.
	Weekly performance summary: structured data sent to Analyst every Monday morning.
	Langfuse integration (mid-Phase B): trace all API write operations and budget decisions. Tag with client_id, campaign_id, decision_reason.

Phase C: Autonomous optimisation (Weeks 7–8: June 16–30)

Phase	Features to build
Phase C	Automatic bid adjustment: increase bids on high-performing ad sets by 10% when ROAS exceeds target. Decrease by 10% when below target. Log every change with reasoning.
	Conversion API push: Analyst sends closed deal data → Media Buyer pushes to Meta and Google via Conversion API to train algorithms on actual buyers.

	Cross-platform budget rebalancing: if Meta outperforms Google this week by more than 20% ROAS gap, shift 10% of budget and alert operator. Dayparting: analyse historical conversion data by hour and day. Adjust ad scheduling to concentrate spend during peak conversion windows.
	Full Langfuse audit: every autonomous decision is a named trace with input data, decision reasoning, and outcome.

Agent 6: The Closer

Lead nurturing, WhatsApp engagement, and appointment booking

Layer: Conversion layer

Stack: LangChain | FastAPI | WhatsApp API | SendGrid/Brevo | Calendly | UV | Langfuse

Role definition

The Closer takes a qualified lead and moves them to a booked appointment. It operates across two channels: WhatsApp (immediate, conversational) and email (structured nurturing sequences). It knows the lead's score, their form answers, and their full CRM history. Every message is personalised from this context.

The Closer does not improvise freely. Every message follows an approved template library set up during client onboarding. The LLM fills personalisation variables and selects the right template for the context. Free-form LLM generation is only used in Phase C when a conversation goes outside the template library: and even then within strict tone and content guidelines.

Template library: required before Phase A can start

The founding team populates this library during client onboarding. The Closer selects from it, it does not invent new approaches without operator approval.

- **First contact SQL:** Sent within 5 minutes of SQL classification. References lead's specific stated problem. Proposes a call.
- **First contact MQL:** Sent within 1 hour. Warmer, less direct. Offers a relevant resource or insight.
- **Follow-up sequence SQL:** 3 messages over 5 days if no response. Each shorter and more direct than the last. Final message offers an easy out.
- **Nurture sequence MQL:** 5 messages over 21 days. Educational content matched to their industry and stated problem. CTA escalates gradually toward booking.
- **Objection handlers:** Pre-approved responses to the top 5 objections per industry: 'I need to think about it', 'Send me more info', 'What's the price?', 'Not the right time', 'I'll discuss with my partner'.
- **Appointment confirmation:** Sent immediately after booking. Includes date, time, video link, and preparation instructions.
- **Rejection message:** Polite, non-dismissive message for leads scored 0–30. Leaves the door open for future re-engagement.
- **Re-engagement:** Sent to MQL leads silent for 7 days. One message only.

Phase A: POC (Week 3: May 12–16)

Goal: Prove the Closer can send a personalised WhatsApp message to a real number and update the CRM. No sequences yet.

Phase

Features to build

Phase A	Telegram interface: operator sends a lead record JSON → Closer returns the first contact message it would send, fully personalised.
	3 hardcoded templates: first contact SQL, first contact MQL, rejection message. No sequences yet.
	WhatsApp Business API connection via 360dialog: send a test message to a real phone number.
	Manual trigger: operator sends 'contact lead_id_123' → Closer selects the correct template, fills personalisation variables, sends to lead's WhatsApp.
	CRM Keeper update: after every send, Closer calls CRM Keeper to update last_action, last_action_at, and messages_sent.
	FastAPI app: Closer exposes POST /whatsapp/incoming to receive inbound WhatsApp reply webhooks from 360dialog or Twilio. Payload validated by Pydantic model. Replies that fail validation are logged and discarded, they do not reach template matching logic. Log every message: { lead_id, channel, template_id, personalisation_variables, sent_at, delivery_status }. README.md and .env.example on day one.

Phase A validation checklist

- Send a SQL lead record → Closer returns correct first contact message personalised with lead's name and stated problem.
- Send a disqualified lead record → Closer returns the rejection message, not the SQL message.
- Trigger a real WhatsApp send to a test number → message arrives on the phone within 30 seconds.
- Check CRM Keeper → last_action, last_action_at, and messages_sent all updated correctly.
- Log file shows correct structure for every send.

Phase B: Full sequences and booking (Weeks 4–6: May 19–June 6)

Phase	Features to build
Phase B	Full email sequences: MQL nurture (5 emails, 21 days) and SQL follow-up (3 emails, 5 days). Triggered automatically by Commander on lead classification.
	WhatsApp reply handling: Closer reads incoming WhatsApp replies and responds using the objection handler library. Match incoming text to nearest objection template.
	Calendar booking integration: Calendly or Cal.com API. When lead agrees to a meeting, Closer sends booking link. On booking confirmation, Closer sends appointment confirmation template and updates CRM. Multi-channel coordination: if lead responds on WhatsApp, pause email sequence. If lead books, stop all sequences. State stored in CRM Keeper. 7-day re-engagement job: scheduled daily check for MQL leads silent for 7 days → send re-engagement message once.

Sequence position tracking: Closer tracks which step of which sequence each lead is on. Stored in CRM notes.

Langfuse integration (mid-Phase B): trace all message sends and template selections. Tag with lead_id, template_id, channel, classification.

Phase C: Intelligent conversation (Weeks 7–8: June 16–30)

Phase	Features to build
Phase C	LLM-generated replies for out-of-template conversations: GPT-4o generates contextual WhatsApp replies within strict tone and content guidelines when no template matches.
	Sentiment detection: analyse incoming reply tone using DeepSeek. Negative or frustrated sentiment → escalate to human operator immediately. Do not continue automated responses. Meeting time optimisation: suggest meeting slots based on lead's time zone and historically best-converting time slots from Analyst data. Post-meeting follow-up: structured follow-up sent after the meeting with next steps. Triggered by calendar event completion webhook.
	30 and 60-day cold lead re-engagement: automated re-engagement for SQL leads that went cold before booking. One message per interval.
	Full Langfuse audit: every LLM-generated reply is a named trace with the input conversation, system prompt used, generated output, and sentiment score.

Agent 7: The Analyst

Performance tracking, ROI analysis, and optimisation signals

Layer: Intelligence layer

Stack: LangChain | CRM API | Ad platform APIs | UV | Langfuse (primary user)

Role definition

The Analyst is the only agent that looks backward. Every other agent acts in the present: the Analyst studies what happened and tells the team what to do differently. It tracks every metric that matters, identifies what is working and what is not, and generates optimisation signals that flow back to the Commander, Media Buyer, and Content & Conversion Strategist.

The Analyst closes the loop. Without it, the system runs on assumptions. With it, every decision is grounded in what actually produced a signed deal. The Analyst also drives the Langfuse evaluation layer: it is the agent most naturally aligned with observability because observation is its entire purpose.

KPI definitions: use these exact definitions, do not deviate

Metric definitions

CPL (Cost Per Lead): Total ad spend / total form submissions. All leads regardless of quality.

CPQ (Cost Per Qualified Lead): Total ad spend / leads with score ≥ 31 (MQL or SQL).

CPQL (Cost Per SQL): Total ad spend / leads with score ≥ 61 .

CPBD (Cost Per Booked Deal): Total ad spend / appointments successfully booked.

ROAS (Return on Ad Spend): Total contract value of closed_won deals / total ad spend. Closed_won only.

Stage conversion rate: % of leads moving from one pipeline stage to the next. Tracked per transition.

Time to first contact: Average minutes between form submission and Closer's first message. Target: under 5 minutes.

Response rate: % of first contact messages receiving a reply within 48 hours.

Meeting show rate: % of booked meetings where the lead actually shows up.

Phase A: Logging only (Weeks 1–3)

The Analyst does not exist as an active agent in Phase A. Its Phase A role is ensuring all other agents log correctly. From week 1, the intern responsible for the Analyst must review every other agent's log output and confirm it matches the mandatory log format. Any agent logging incorrectly is flagged before it reaches Phase B.

Phase	Features to build
Phase A	No agent built yet: Phase A for the Analyst is audit duty.

Review log output from Commander, CRM Keeper, and Qualifier as each completes Phase A.
Confirm every log entry contains all mandatory fields: agent_name, action_type, input_summary, output_summary, lead_id, client_id, model_used, latency_ms, timestamp.
Document any logging gaps and raise them before the offending agent moves to Phase B.
Set up the central log store (Supabase or Postgres) that all agents will write to in Phase B.

Phase A validation checklist

All Phase A agents (Commander, CRM Keeper, Qualifier) are logging with correct mandatory structure.

Central log store (Supabase/Postgres) is running and accepting writes.

Log ingestion script tested: reads agent JSON logs and writes them to the central store correctly.

Phase B: KPI dashboard and alerts (Weeks 4–6: May 19–June 6)

Phase	Features to build
Phase B	Data pull: Analyst reads from central log store and CRM Keeper to calculate all 9 defined KPIs.
	Telegram report: operator sends 'report' → Analyst returns a formatted KPI summary with values and week-on-week change.
	Anomaly detection: if any stage conversion rate drops below 50% of the previous week's rate, send an immediate Telegram alert.
	Attribution by source: break down CPL, CPQ, and CPQL by campaign, ad set, and creative asset. Identify the top and bottom performer for each.
	Landing page performance: track conversion rate per landing page. Flag any page below 15% visitor-to-form rate to Content Strategist.
	Weekly optimisation report: generated every Monday. Specific recommended actions: scale X campaign, pause Y ad set, rewrite Z copy asset. Sent to Commander.
	Langfuse integration (from the start of Phase B): Analyst is the primary Langfuse user. All other agent traces are reviewed here. Set up weekly eval jobs for Qualifier accuracy and Content Strategist output quality.

Phase C: Predictive feedback loop (Weeks 7–8: June 16–30)

Phase	Features to build
Phase C	Conversion API push: Analyst compiles all closed_won deals from CRM and instructs Media Buyer to push them to Meta and Google via Conversion API. Runs weekly.

	Predictive ROAS modelling: based on current pipeline (leads per stage, historical stage-to-stage conversion rates), project expected revenue for the next 30 days. Scoring model feedback: Analyst compares Qualifier's SQL predictions against actual closed deals. Sends calibration signals to Qualifier to adjust dimension weights. Logs every adjustment. A/B test conclusions: when a Content Strategist A/B test reaches statistical significance (minimum 100 conversions per variant), Analyst flags the winner and instructs the Content Strategist to deprecate the loser. Cohort analysis: track lead quality by acquisition date, campaign, and audience segment. Identify which sources produce the best long-term conversion outcomes.
	Full Langfuse evaluation suite: all 6 agents have active eval jobs running in Langfuse. Analyst reviews eval results weekly and surfaces issues to the Commander.

4. Global Rules: Apply to Every Agent

4.1 Mandatory log format

Every agent logs every action in this exact JSON structure

```
{
  agent_name: string,
  action_type: string (e.g. 'score_lead', 'send_message', 'route_task', 'update_crm'),
  input_summary: string (brief description of what the agent received: not the full payload),
  output_summary: string (brief description of what the agent produced),
  lead_id: string or null (null for non-lead-specific actions),
  client_id: string,
  model_used: string (e.g. 'gpt-4o', 'deepseek-chat', 'local-phi3', 'rule-based'),
  latency_ms: integer (total time for this action in milliseconds),
  timestamp: ISO 8601 string
}
```

Phase A: write to local JSON file (one file per agent, append-only).

Phase B: write to central Postgres/Supabase table. Keep local file as backup.

Phase C: write to central store AND emit as a Langfuse event within the active trace.

4.2 Error handling

- Every agent must handle all errors explicitly. Silent failures are unacceptable.
- On any error: log the error (same mandatory format, add `error_type` and `error_message` fields), then notify the Commander.
- The Commander decides whether to retry, reassign, or escalate to the operator.
- No agent retries more than 2 times independently. After 2 failures, escalate to Commander.
- API rate limits are expected, not errors. Every agent implements exponential backoff with jitter for all external API calls.
- Never expose raw API error messages to the operator. Translate to a plain-language description of what failed and what the agent will do next.

4.3 Model selection rules

- **Local SLM (Phi-3, Mistral 7B via Ollama):** Simple classification, keyword extraction, form field parsing, template variable filling. Never for nuanced reasoning or copy generation.
- **DeepSeek API:** Lead scoring reasoning, routing decisions, objection classification, content strategy. Good cost-to-capability ratio for medium-complexity tasks.
- **OpenAI GPT-4o:** Ad copy generation, email drafting, daily digest writing, complex WhatsApp conversations, score explanations. Highest cost: use only where quality is the primary concern.

4.4 Documentation checklist: every agent at every phase

- README.md created on day one before any logic is written.
- All functions and classes have complete Google-style docstrings before Phase A sign-off.
- .env.example contains every required environment variable with a placeholder value and a comment explaining what it is.
- CHANGELOG.md updated at the end of every phase with a plain-English description of what was built and what was validated.
- Inline comments added to all non-obvious logic, all hardcoded values, all API calls, and all error handlers.
- Test files exist in the tests/ directory with at least one test per public function before Phase B sign-off.

4.5 MVP definition: what done looks like on June 30

The MVP is complete when all of the following are simultaneously true:

13. A new lead enters via a form, gets scored by the Qualifier, receives a personalised WhatsApp message within 5 minutes, and has an appointment booked: all without human intervention.
14. The Commander receives an operator command via Telegram and routes it correctly every time.
15. The Analyst produces a weekly performance report with correct KPI values verified against raw data.
16. The above runs on at least one real client deployment, not a test environment.
17. All agents have complete Phase A and Phase B validation checklists signed off by the responsible intern.
18. Langfuse is active for all agents. All LLM calls are traced. The Analyst's weekly eval jobs are running.
19. Every agent has a complete README.md, full docstring coverage, and an up-to-date CHANGELOG.md.

Final note for interns

Read this document fully before writing a single line of code. If something is unclear, ask before building.

Do not skip the validation checklists. A feature that works in isolation often fails in the full flow.

Do not move to Phase B of any agent until Phase A is fully signed off.

Build less and validate more. A solid Phase A is always better than a half-built Phase B.

Documentation is not a task you do at the end. It is something you write as you build.

Log everything from day one. Retroactively adding logging is always incomplete.

When something breaks, log it, escalate it to the Commander flow, and document what caused it in the CHANGELOG.